Desenvolvimento de Sistemas para Dispositivos Móveis

Camadas, Koin, dados reais, cache e testes de ViewModel

DIM0524 — Turma 01 · Sprint 2 · aula em vídeo

Prof. Fernando · UFRN · 2026.2

Onde paramos

```
  passo 11   o estado no ViewModel, com StateFlow
  passo 12   carregar numa corrotina; os quatro estados como classe selada
```

Passo	Hoje
13	Camadas (domínio, dados, apresentação) e injeção de dependência com Koin
14	Dados reais: o app chama a API de tarefas com o Ktor Client
15	Abrir sem rede: a última lista guardada no aparelho, com DataStore
16	Testes de ViewModel com Turbine e dublês

Branch `aula`: `./passo.sh 12`. Para os passos 14 e 15, a API de tarefas precisa estar no ar:

```
docker compose up --build    # sobe a API de tarefas, o banco e o serviço Go
```

Passo 13

Camadas e injeção de dependência com Koin

Três camadas, em pacotes

```
br.ufrn.exemplo.tarefas
├── App.kt, Modulos.kt        tema, navegação e a raiz de composição
├── dominio/                  Tarefa, regras, a porta RepositorioDeTarefas
├── dados/                    as implementações do repositório (memória, rede, cache)
└── apresentacao/             TarefasViewModel, EstadoTela, as telas
```

Camada	Pode depender de	Não pode depender de
domínio	só Kotlin	Compose, Ktor, DataStore, Koin
dados	domínio, bibliotecas de rede e de disco	apresentação
apresentação	domínio, Compose, ViewModel	detalhes de dados (Ktor, DataStore)

- O ViewModel conhece a interface `RepositorioDeTarefas`, e não quem a implementa
- É o critério "Estado e arquitetura": domínio sem Compose nem infraestrutura, repositórios por interface, sem lógica de negócio dentro de `@Composable`

Injeção de dependência, sem biblioteca

```
class TarefasViewModel(private val repositorio: RepositorioDeTarefas) : ViewModel()

// Alguém precisa montar a cadeia inteira, na ordem certa:
val cliente = criarClienteHttp()
val cache = CacheDeTarefas(criarDataStore(diretorio))
val repositorio = RepositorioComCache(RepositorioRemoto(cliente, endereco), cache)
val vm = TarefasViewModel(repositorio)
```

- **Injeção de dependência:** a classe recebe o que usa, em vez de criar. O ViewModel não faz `RepositorioRemoto(...)` por conta própria
- Ganho: o teste passa um dublê pelo mesmo construtor
- Custo: alguém tem de montar a cadeia, e cada objeto deve ser criado uma vez só (um cliente HTTP, um DataStore)
- O Koin é esse "alguém": a montagem vira uma lista de definições, e ele resolve a ordem

Koin: quem monta as peças

```
// Modulos.kt: o único lugar que conhece as três camadas
val moduloTarefas = module {
    single<RepositorioDeTarefas> { RepositorioEmMemoria() }
    viewModelOf(::TarefasViewModel)          // o Koin passa o repositório ao construtor
}
```

```
class TarefasViewModel(private val repositorio: RepositorioDeTarefas) : ViewModel()   // recebe, não cria

@Composable
fun App() {
    KoinApplication(application = { modules(moduloTarefas) }) { ... }
}

@Composable
fun Conteudo(largo: Boolean, viewModel: TarefasViewModel = koinViewModel()) { ... }
```

- Trocar a implementação (memória → rede) é trocar uma linha no módulo
- Nos testes, o ViewModel é criado com `new`, com o repositório que o teste quiser: sem Koin
- `ModulosTest` confere que o módulo consegue montar o ViewModel

📖 **Ref.** Koin — Compose Multiplatform

O vocabulário do Koin

Peça	O que faz
`module { }`	uma lista de definições: como criar cada coisa
`single { ... }`	cria na primeira vez que alguém pede e reaproveita depois
`factory { ... }`	cria uma instância nova a cada pedido
`get()`	dentro de uma definição: pede ao Koin uma dependência já declarada
`viewModelOf(::TarefasViewModel)`	ViewModel com o ciclo de vida certo; o construtor recebe o que pede
`KoinApplication(application = { modules(...) }) { }`	liga o Koin à interface, em `App()`
`koinViewModel()`	numa função `@Composable`: pede o ViewModel

> Erro comum: esquecer uma definição. O app compila e só quebra ao abrir a tela: "No definition found for type RepositorioDeTarefas".

Conferir o módulo num teste

```
// O módulo do Koin consegue montar o ViewModel? Se faltar uma definição, o teste acusa
// aqui, e não com o app aberto.
class ModulosTest {
    @BeforeTest fun trocarMain() = Dispatchers.setMain(UnconfinedTestDispatcher())
    @AfterTest fun restaurarMain() = Dispatchers.resetMain()

    @Test
    fun oModuloMontaOViewModel() {
        val koin = koinApplication { modules(moduloTarefas(SystemTemporaryDirectory.toString())) }.koin
        assertNotNull(koin.get<TarefasViewModel>())
        koin.close()
    }
}
```

- Os outros testes não usam o Koin: criam o ViewModel com o dublê que querem
- Este é o único que liga as peças de verdade, e transforma o erro de execução em erro de teste

Passo 14

Dados reais com o Ktor Client

DTO separado da entidade

```
// dados/: o formato do JSON da API
@Serializable
data class TarefaDto(val id: Int, val titulo: String, val feita: Boolean = false)

fun TarefaDto.paraDominio() = Tarefa(id = id, titulo = titulo, feita = feita)
```

```
class RepositorioRemoto(private val cliente: HttpClient, private val enderecoDaApi: String) : RepositorioDeTarefas {

    override suspend fun listar(): List<Tarefa> = traduzindoFalhas {
        cliente.get("$enderecoDaApi/tarefas").body<List<TarefaDto>>().map { it.paraDominio() }
    }

    override suspend fun adicionar(titulo: String): Tarefa = traduzindoFalhas {
        cliente.post("$enderecoDaApi/tarefas") {
            contentType(ContentType.Application.Json)
            setBody(NovaTarefaDto(titulo))
        }.body<TarefaDto>().paraDominio()
    }
}
```

Se a API mudar um nome de campo, mudam o DTO e a conversão; o domínio e as telas continuam iguais.

📖 **Ref.** Ktor — Client

O cliente e o repositório remoto

```
fun criarClienteHttp(): HttpClient = HttpClient {     // sem motor: OkHttp no Android, CIO no desktop
    expectSuccess = true                              // 4xx e 5xx viram exceção
    install(ContentNegotiation) {
        json(Json { ignoreUnknownKeys = true })       // campos novos na API não quebram o app
    }
}

class RepositorioRemoto(private val cliente: HttpClient, private val enderecoDaApi: String) : RepositorioDeTarefas {
    override suspend fun listar(): Listagem = traduzindoFalhas {
        Listagem(cliente.get("$enderecoDaApi/tarefas").body<List<TarefaDto>>().map { it.paraDominio() })
    }

    override suspend fun adicionar(titulo: String): Tarefa = traduzindoFalhas {
        cliente.post("$enderecoDaApi/tarefas") {
            contentType(ContentType.Application.Json)
            setBody(NovaTarefaDto(titulo))
        }.body<TarefaDto>().paraDominio()
    }
}
```

São as rotas `GET` e `POST /tarefas` da API de tarefas.

Erros de rede com nome

```
sealed class FalhaNoRepositorio(mensagem: String) : Exception(mensagem) {      // no domínio
    class SemConexao : FalhaNoRepositorio("sem conexão com o servidor")
    class RespostaInvalida(detalhe: String) : FalhaNoRepositorio("resposta inválida: $detalhe")
}
```

O que aconteceu	Exceção do Ktor	Falha do domínio	Na tela
API fora do ar, sem rede	`IOException`	`SemConexao`	"Sem conexão com o servidor. Confira a rede e tente de novo."
status 4xx ou 5xx	`ResponseException` (`expectSuccess = true`)	`RespostaInvalida`	"O servidor respondeu algo inesperado (...)"
JSON fora do formato	`ContentConvertException`	`RespostaInvalida`	idem

- Cada falha tem um estado de tela próprio: é o que a rubrica pede em "Dados reais e erro de rede"
- O ViewModel não conhece o Ktor: só as falhas do domínio
- Salvar que falha não apaga a lista: vira um aviso acima dela

Traduzir as exceções da biblioteca

```
// Exceções da biblioteca viram falhas do domínio: o ViewModel não precisa conhecer o Ktor.
private suspend fun <T> traduzindoFalhas(chamada: suspend () -> T): T = try {
    chamada()
} catch (e: IOException) {
    throw FalhaNoRepositorio.SemConexao()            // servidor fora, sem rede, recusado
} catch (e: ResponseException) {
    throw FalhaNoRepositorio.RespostaInvalida("status ${e.response.status.value}")
} catch (e: ContentConvertException) {
    throw FalhaNoRepositorio.RespostaInvalida("JSON fora do formato")
} catch (e: SerializationException) {
    throw FalhaNoRepositorio.RespostaInvalida("JSON fora do formato")
}
```

```
// no ViewModel: salvar que falha vira um aviso, e a lista continua na tela
} catch (e: FalhaNoRepositorio) {
    _estado.update { it.copy(aviso = "A tarefa não foi salva: ${e.message}.") }
}
```

- `chamada: suspend () -> T` é uma função como parâmetro: o mesmo tratamento serve para `listar` e `adicionar`
- O `catch` pega só as falhas do domínio: o cancelamento segue adiante sozinho

Onde está a API

```
expect val enderecoDaApi: String?                    // commonMain; nulo = sem API, repositório em memória

actual val enderecoDaApi: String? = when (val valor = System.getenv("TAREFAS_API")) {   // desktop
    null, "" -> "http://localhost:8080"
    "memoria" -> null                                 // TAREFAS_API=memoria: roda sem a API
    else -> valor
}
actual val enderecoDaApi: String? = "http://10.0.2.2:8080"   // Android (emulador)
```

Onde o app roda	Endereço da API
desktop, na mesma máquina	`http://localhost:8080`
emulador Android	`http://10.0.2.2:8080`: o computador que roda o emulador
sem a API (Codespace, laboratório)	`TAREFAS_API=memoria` ou `mise run app:sem-api`: repositório em memória
celular na mesma rede Wi-Fi	o IP do computador, por exemplo `http://192.168.0.10:8080`

No Android: permissão `INTERNET` e, para `http://` local, `usesCleartextTraffic="true"` (num app publicado, a API usa `https` e essa linha sai).

Demonstração: a API de tarefas no ar, `./passo.sh 14`, `mise run app`: as tarefas vêm do banco. Pare a API e toque em **Tentar de novo**.

Testar o repositório sem a API

```
// O repositório contra um servidor HTTP de mentira (MockEngine): sem rede, sem a API.
private fun repositorio(resposta: MockEngine) = RepositorioRemoto(
    HttpClient(resposta) { expectSuccess = true; install(ContentNegotiation) { json() } },
    enderecoDaApi = "http://api",
)

@Test
fun converteOJsonDaApiEmTarefas() = runTest {
    val repo = repositorio(MockEngine { respond("""[{"id":1,"titulo":"Estudar Ktor","feita":true}]""", headers = json) })
    assertEquals("Estudar Ktor", repo.listar().tarefas.single().titulo)
}

@Test
fun servidorForaViraSemConexao() = runTest {
    val repo = repositorio(MockEngine { throw IOException("conexão recusada") })
    assertFailsWith<FalhaNoRepositorio.SemConexao> { repo.listar() }
}
```

- O `MockEngine` responde o que o teste mandar: JSON certo, status 500, JSON fora do formato, conexão recusada
- São os 4 testes de `RepositorioRemotoTest`, e rodam sem a API de tarefas

Passo 15

Abrir sem rede

Um repositório que usa outro

```
class RepositorioComCache(
    private val remoto: RepositorioDeTarefas,
    private val cache: CacheDeTarefas,
) : RepositorioDeTarefas {

    override suspend fun listar(): Listagem = try {
        remoto.listar().also { cache.salvar(it.tarefas) }          // guarda cada lista recebida
    } catch (e: FalhaNoRepositorio.SemConexao) {
        val guardadas = cache.ler() ?: throw e                      // nunca carregou: o erro continua
        Listagem(guardadas, doCache = true)
    }
}
```

```
class CacheDeTarefas(private val dataStore: DataStore<Preferences>) {
    suspend fun salvar(tarefas: List<Tarefa>) {
        dataStore.edit { it[chave] = Json.encodeToString(tarefas.map { TarefaDto(it.id, it.titulo, it.feita) }) }
    }
    suspend fun ler(): List<Tarefa>? = dataStore.data.first()[chave]?.let { ... }
}
```

- O ViewModel não sabe que existe cache: só recebe `doCache = true`, e a tela mostra o aviso
- Para listas grandes ou consultas, um banco no aparelho (Room ou SQLDelight); para uma lista pequena, o DataStore basta

📖 **Ref.** DataStore — Kotlin Multiplatform

O cache com DataStore

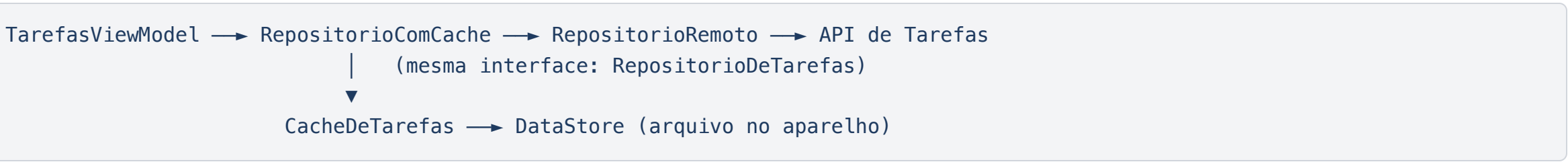
```
// A última lista recebida, guardada num arquivo do aparelho. Sobrevive ao fechamento do app.
class CacheDeTarefas(private val dataStore: DataStore<Preferences>) {
    private val chave = stringPreferencesKey("ultima_lista")

    suspend fun salvar(tarefas: List<Tarefa>) {
        val dtos = tarefas.map { TarefaDto(it.id, it.titulo, it.feita) }
        dataStore.edit { it[chave] = Json.encodeToString(dtos) }
    }

    suspend fun ler(): List<Tarefa>? =
        dataStore.data.first()[chave]?.let { json -> Json.decodeFromString<List<TarefaDto>>(json).map { it.paraDominio() } }
}
```

- O DataStore guarda pares chave e valor num arquivo, de forma assíncrona: `edit` e `data` não travam a tela
- `data` é um `Flow`: `first()` lê o valor atual uma vez
- A lista vai como o JSON dos DTOs, o mesmo formato que vem da API

Um repositório que envolve outro



Situação	O que `listar()` devolve
a API respondeu	a lista nova, que também é guardada
sem conexão, com algo guardado	a última lista, com `doCache = true`
sem conexão, nada guardado	a falha `SemConexao` continua: a tela mostra o erro
resposta inválida	a falha continua: o cache só cobre a falta de conexão

- Um objeto que implementa a mesma interface que recebe e acrescenta um comportamento: o padrão **Decorator**
- O ViewModel não muda: o Koin entrega o repositório com cache no lugar do remoto

Onde fica o arquivo

```
fun moduloTarefas(diretorioDeDados: String) = module {
    single { criarClienteHttp() }
    single { CacheDeTarefas(criarDataStore(diretorioDeDados)) }        // um DataStore por arquivo
    single<RepositorioDeTarefas> {
        val endereco = enderecoDaApi
        if (endereco == null) RepositorioEmMemoria()
        else RepositorioComCache(RepositorioRemoto(get(), endereco), get())
    }
    viewModelOf(::TarefasViewModel)
}
```

Plataforma	App(diretorioDeDados = ...)
Android (`MainActivity`)	`filesDir.path`, a pasta privada do app
desktop (`main.kt`)	`~/.tarefas-compose`

Demonstração: abrir o app com a API no ar; fechar; parar a API (`docker compose stop api-ktor`); abrir de novo: "Sem conexão: mostrando a última lista salva."

Onde guardar dados no aparelho

Opção	Para quê	No projeto
DataStore (Preferences)	pares chave e valor; configurações; um cache pequeno	o exemplo guarda a última lista numa chave só
Room	banco SQLite com consultas conferidas na compilação; Kotlin Multiplatform	listas grandes, buscas, relações entre dados
SQLDelight	você escreve o SQL, e ele gera o código Kotlin; Kotlin Multiplatform	idem, para quem prefere escrever o SQL

Dois cuidados com o DataStore:

- **Um DataStore por arquivo, por processo.** Dois para o mesmo arquivo é erro; o Koin cria um só (`single`)
- **Cada plataforma tem a sua pasta**: o diretório chega de fora, como parâmetro de `App`

A rubrica pede que o app abra sem rede mostrando o último conteúdo; a forma de guardar fica a critério do grupo.

Passo 16

Testes de ViewModel com Turbine

Um dublê e o relógio virtual

```
class RepositorioFalso(var listar: () -> Listagem = { Listagem(listOf(Tarefa(1, "Estudar Flow"))) }) : RepositorioDeTarefas {
    override suspend fun listar(): Listagem { delay(500); return listar.invoke() }   // 500 ms de "rede"
    ...
}
```

```
@Test
fun carregarPassaPorCarregandoEChegaASucesso() = runTest {
    Dispatchers.setMain(StandardTestDispatcher(testScheduler))   // o Main usa o relógio do teste
    TarefasViewModel(RepositorioFalso()).estado.test {           // Turbine: observa o StateFlow
        assertEquals(EstadoTela.Carregando, awaitItem().tela)    // o estado intermediário aparece
        assertIs<EstadoTela.Sucesso>(awaitItem().tela)           // e os 500 ms passam sem esperar
    }
}
```

- `awaitItem()` é o próximo estado emitido, na ordem: o teste vê a sequência, e não só o fim
- O tempo é virtual: os 500 ms do dublê passam na hora

📖 **Ref.** Turbine · Android — Testing coroutines

Turbine: a sequência inteira

```
@Test
fun tentarDeNovoDepoisDoErro() = runTest {
    usarRelogioDoTeste()
    val repositorio = RepositorioFalso { throw FalhaNoRepositorio.SemConexao() }
    val vm = TarefasViewModel(repositorio)
    vm.estado.test {
        skipItems(1)                                        // Carregando
        assertIs<EstadoTela.Erro>(awaitItem().tela)
        repositorio.listar = { Listagem(listOf(Tarefa(1, "Voltou"))) }
        vm.carregar()
        assertEquals(EstadoTela.Carregando, awaitItem().tela)
        assertIs<EstadoTela.Sucesso>(awaitItem().tela)
    }
}
```

- O dublê muda de comportamento no meio do teste: primeiro falha, depois responde
- `skipItems(1)` pula o que não interessa; `awaitItem()` falha se o estado esperado não chegar
- No fim do bloco, o Turbine confere que não sobrou estado sem verificação

Dois despachantes de teste

	`StandardTestDispatcher(testScheduler)`	`UnconfinedTestDispatcher()`
quando a corrotina roda	entra numa fila e roda quando o teste espera	na hora, até a primeira suspensão
no exemplo	testes de ViewModel com Turbine: é possível ver o `Carregando`	testes de tela e o `ModulosTest`: só o resultado importa

```
private fun TestScope.usarRelogioDoTeste() = Dispatchers.setMain(StandardTestDispatcher(testScheduler))

@AfterTest
fun restaurarMain() = Dispatchers.resetMain()
```

> Esquecer o `resetMain` faz o próximo teste herdar o despachante do anterior: falhas que dependem da ordem dos testes.

A suíte, no fim do Passo 16

Arquivo	Testes	O que cobre
`TarefasViewModelTest`	7	carregando → sucesso, vazio, sem conexão, resposta inválida, tentar de novo, lista do cache, adicionar
`TelasTest`	7	formulário, linha alternável, cabeçalho, navegação, janela larga, erro com tentar de novo, aviso de cache
`RepositorioRemotoTest`	4	JSON → tarefas, servidor fora, 500, JSON fora do formato (`MockEngine`)
`RepositorioComCacheTest`	3	guarda, devolve sem conexão, erro sem nada guardado
`RegrasTest` e `ModulosTest`	4	regras puras e o módulo do Koin

```
./gradlew :composeApp:desktopTest      # 25 testes, no alvo desktop, sem emulador
```

A rubrica pede ≥ 5 de interface e ≥ 5 de ViewModel, verdes no CI.

Para conferir o entendimento

1. Por que o ViewModel recebe `RepositorioDeTarefas`, e não `RepositorioRemoto`?
2. Qual a diferença entre `single` e `factory` no Koin, e por que o DataStore é `single`?
3. O que acontece com uma resposta 500 se o cliente não tiver `expectSuccess = true`?
4. Por que a falta de conexão usa o cache e a resposta inválida não?
5. O que o Turbine mostra que um `assertEquals` no estado final não mostra?

O que a Sprint 2 cobra

Critério	Peso	O que mostrar
Estado e arquitetura	25%	ViewModel com `StateFlow`; camadas em `commonMain`; domínio sem Compose nem infraestrutura; repositórios por interface
Estados da interface	15%	carregando, erro, vazio e sucesso, por classe selada, na tela
Dados reais e erro de rede	25%	API real, Ktor Client, DTO separado; falta de conexão e resposta inválida com estado próprio
Persistência local	10%	o app abre sem rede mostrando o último conteúdo
Testes	15%	≥ 5 de interface e ≥ 5 de ViewModel com Turbine e dublês, verdes no CI
App e documentação	10%	sem crash; `docs/arquitetura.md`; README; APK numa release

Prova em **11/11**; *daily meetings* em 23 e 25/11; **entrega em 30/11**.

Onde estudar depois

- Leitura da sprint: `leituras/moveis-s2.md`, capítulos 5 a 9
- Passos: `exemplos/tarefas-compose/PASSOS.md` · branch `aula`, tags `passo-13` a `passo-16`
- Koin — Compose Multiplatform
- Ktor — Client
- DataStore — Kotlin Multiplatform
- Turbine · Android — Testing coroutines